

密 级： 内部

阶 段： 需求分析阶段

版 次： V1.0

TH-Scheduler-周期任务调度器 软件需求规格说明

产品型号-Scheduler

共 XX 页

太行项目组

2026-05-06

TH-Scheduler-周期任务调度器

软件需求规格说明

产品型号-Scheduler

编制：_____日期：_____

校对：_____日期：_____

审核：_____日期：_____

标审：_____日期：_____

审定：_____日期：_____

批准：_____日期：_____

目 录

1 范围	1
1.1 标识	1
1.2 系统概述.....	1
1.3 文档概述.....	1
2 引用文档	1
3 需求	2
3.1 要求的状态和方式	2
3.1.1 初始化状态	2
3.1.2 正常运行状态	2
3.1.3 超时告警状态	2
3.2 软件能力需求	2
3.2.1 高精度定时器功能	2
3.2.2 时间漂移补偿功能	3
3.2.3 电流序列管理功能	3
3.2.4 主循环调度功能	3
3.2.5 降采样输出功能	4
3.3 软件外部接口需求	4
3.3.1 与电池模型的接口	4
3.3.2 与标准输出的接口	4
3.4 软件内部接口需求	4
3.4.1 模块间接口规范	5
3.4.2 模块内部接口规范	5
3.5 软件内部数据需求	5
3.6 适应性需求	6
3.7 安全性需求	6
3.8 保密性需求	6

3.9 软件环境需求	6
3.10 计算机资源需求	6
3.10.1 计算机硬件需求	6
3.10.2 计算机硬件资源使用需求	6
3.10.3 计算机软件需求	7
3.10.4 计算机通信需求	7
3.11 软件质量因素	7
3.12 设计和实现约束	7
3.13 人员需求	8
3.14 培训需求	8
3.15 软件保障需求	8
3.16 其他需求	8
3.17 验收、交付和包装需求（修改有关内容）	8
3.18 需求的优先顺序和关键程度	8
4 合格性规定	8
5 需求可追踪性	9
6 注释	10

1 范围

1.1 标识

本文档适用于周期性任务调度器软件（Scheduler），标识号为 TH/Scheduler-SRS-V1.0，版本号为 V1.0，发布号为 Release 1。该软件由太行项目组嵌入式软件研发部开发，用于嵌入式 Linux 系统中驱动电池仿真模型以固定周期运行。

- a) 文档标识号：TH/Scheduler-SRS-V1.0；
- b) 标题：软件需求规格说明
- c) 软件名称：TH-Scheduler-周期任务调度器
- d) 软件缩写：Scheduler
- e) 软件版本号：V1.0。

1.2 系统概述

周期性任务调度器软件是一个用于嵌入式 Linux 系统的实时任务调度器，以 25ms 固定周期驱动电池模型（battery_model）运行。软件基于 Linux 高精度定时器（clock_nanosleep）实现周期调度，具备时间漂移补偿能力，确保长时间运行的平均周期精度满足要求。

该软件应用于嵌入式电池管理系统（BMS）场景，为电池仿真模型提供精确的时序驱动。开发背景源于太行项目对 ARM Linux（内核 4.1.15，单核）平台上实时电池仿真的需求。调度器负责初始化电池模型、模拟放电电流序列、周期性执行模型步进、记录并打印输出结果。

软件的主要特性包括：基于 clock_nanosleep 的绝对时间补偿机制（保证长时间运行无明显漂移）、支持恒流和脉冲电流测试序列、超时检测与告警机制、降采样输出（每 1 秒打印一次）。调度器不依赖 RTOS，在标准嵌入式 Linux 环境下运行。

1.3 文档概述

本文档规定了周期性任务调度器软件的需求规格，包括功能需求、性能需求、接口需求、环境需求和质量需求等方面。文档内容涵盖软件的范围、引用文档、详细需求说明、合格性规定、需求可追踪性和相关注释。

本文档为软件的设计、开发、测试和验收提供依据，适用于嵌入式软件开发人员、测试人员、系统集成人员和项目管理人员使用。本文档为内部文档，仅限太行项目组内部使用，不得对外传播。

2 引用文档

本章列出周期性任务调度器软件需求规格说明编制所引用的标准和相关文档。

表 2-1 引用文件

序号	名称	版本	发布日期	来源
1	《GJB 438C 软件文档编制规范》	438C	2024-01-01	相关标准库
2	《电池仿真模型软件需求规格说明》	V1.0	2026-05-06	太行项目组

3 需求

3.1 要求的状态和方式

调度器软件在以下状态下运行：

3.1.1 初始化状态

软件在初始化状态下运行，完成电池模型初始化（调用 `battery_init()`）、定时器初始化（设定 25ms 周期）、电流序列加载等准备工作。初始化完成后转入正常运行状态。

3.1.2 正常运行状态

软件在正常运行状态下运行，主循环以 25ms 周期执行：获取当前电流值→调用 `battery_step()`→读取电池状态→降采样输出→等待下一周期。软件在此状态下应满足所有功能需求和性能需求。

3.1.3 超时告警状态

当某一周期的任务执行时间超过 25ms 时，软件进入超时告警状态。在此状态下，调度器跳过本次延迟等待，打印超时警告信息，立即进入下一周期。超时告警不影响后续正常调度。

3.2 软件能力需求

调度器软件的核心能力需求包括以下功能模块：

3.2.1 高精度定时器功能

功能名称：高精度定时器

功能描述：封装 Linux 高精度定时器，提供基于 CLOCK_MONOTONIC 的绝对时间等待机制。记录第一次调用的起始时间，后续每次计算下一个绝对唤醒时间，确保平均周期为 25ms。采用 `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...)` 实现绝对时间等待。

输入：定时周期（`interval_ms`，默认 25ms）

输出：无（阻塞等待到下一个周期起点）

性能要求：单次定时误差不超过 $\pm 2\text{ms}$

约束条件：使用 CLOCK_MONOTONIC 时钟源，不受系统时间调整影响

3.2.2 时间漂移补偿功能

功能名称：时间漂移补偿

功能描述：通过绝对时间补偿机制消除长时间运行的时间漂移。每个周期计算下一个绝对唤醒时间（ $\text{next_wake} = \text{start_time} + n * \text{interval}$ ），即使某次任务执行超时，后续周期仍能追踪到正确的绝对时间点。连续运行 1000 个周期后总时间误差不超过 50ms。

输入：当前周期编号、起始时间、周期长度

输出：下一个唤醒时间点

性能要求：补偿计算时间不超过 0.01ms

约束条件：超时时打印警告但继续运行

3.2.3 电流序列管理功能

功能名称：电流序列管理

功能描述：管理测试电流序列，支持按时间段切换不同的电流模式。预设测试序列：前 10 秒恒流 10A，接着 5 秒脉冲 50A（每 0.5 秒切换一次），最后持续放电至 SOC 接近 0。

输入：当前仿真时间（s）

输出：当前应输出的电流值（A）

性能要求：查询时间不超过 0.01ms

约束条件：电流序列可通过数组配置，支持硬编码

3.2.4 主循环调度功能

功能名称：主循环调度

功能描述：实现调度器主循环，按 25ms 周期驱动电池模型运行。主循环流

程：初始化→获取当前电流→调用 `battery_step(I_out, DT)`→读取电池状态→每 1 秒打印一行输出→等待下一周期。

输入：无（由 `main` 函数启动）

输出：时间(s)、SOC、电压(V)、功率(W)的格式化输出

性能要求：单循环执行时间不超过 5ms（含电池模型计算）

约束条件：不修改电池模型内部结构体，通过 `getter` 函数访问

3.2.5 降采样输出功能

功能名称：降采样输出

功能描述：将 25ms 周期的内部数据降采样为 1 秒间隔的输出，减少输出数据量。每 40 个周期（1 秒）打印一行数据，输出格式为'时间(s) SOC 电压(V) 功率(W)'。

输入：当前周期计数、电池状态数据

输出：格式化的标准输出行

性能要求：打印操作不阻塞超过 1ms

约束条件：最后一秒的数据即使不足 40 个周期也应输出

3.3 软件外部接口需求

调度器软件的接口需求如下：

3.3.1 与电池模型的接口

调度器通过函数调用接口与电池模型交互，调用 `battery_init()`进行初始化，每 25ms 调用 `battery_step(I_out, DT)`执行一步计算，通过 `battery_get_voltage()`、`battery_get_soc()`、`battery_get_ploss()`读取模型输出。调度器不得修改电池模型的内部结构体。

3.3.2 与标准输出的接口

调度器通过标准输出（`stdout`）打印运行结果，每 1 秒输出一行格式化数据：时间(s) SOC 电压(V) 功率(W)。超时告警信息也通过标准错误输出（`stderr`）打印。不涉及网络通信或文件输出。

3.4 软件内部接口需求

调度器软件内部接口需求如下：

1. 定时器模块与主循环接口：定时器模块通过 `timer_init()`和 `timer_wait_next()`函数向主循环提供定时服务。`timer_init()`设定周期参数，`timer_wait_next()`阻塞到

下一个周期起点。

2. 电流序列模块与主循环接口：电流序列模块通过 `get_current_at_time(t)` 函数向主循环提供当前电流值查询服务，输入为仿真时间，输出为电流值。

3. 主循环与电池模型接口：主循环通过 `battery_model.h` 中声明的函数接口调用电池模型，不直接访问其内部数据结构。

3.4.1 模块间接口规范

调度器内部模块间接口规范如下：

1. 接口定义：每个模块通过头文件（.h）声明对外接口函数，实现文件（.c）中定义具体实现。

2. 错误处理：`clock_gettime` 和 `clock_nanosleep` 调用失败时，定时器模块应打印错误信息并采取降级策略（如使用忙等待）。

3. 数据传递：模块间数据传递通过函数参数和返回值完成，不使用全局变量（除必要的定时器状态外）。

4. 模块解耦：定时器模块、电流序列模块与电池模型完全解耦，仅主循环模块依赖其他三个模块。

3.4.2 模块内部接口规范

调度器内部模块内接口规范如下：

1. 函数接口：模块内部函数遵循统一的命名规范（模块名_功能名），参数和返回值类型明确。

2. 数据结构：定时器模块内部使用 `timer_state_t` 结构体存储起始时间、周期和下次唤醒时间等状态。

3. 头文件保护：所有头文件使用 `include guard` 宏保护，命名格式为 `__MODULE_NAME_H__`。

3.5 软件内部数据需求

调度器软件内部数据需求如下：

1. 定时器状态结构体（`timer_state_t`）：存储起始时间（`struct timespec`）、周期间隔（`int`，单位 `ms`）、下次唤醒时间（`struct timespec`）。

2. 电流序列数组：存储测试电流序列的时间-电流对，采用结构体数组表示，每个元素包含开始时间和电流值。

3. 运行统计变量：周期计数器（`int`）、超时计数器（`int`）、总运行时间（`double`）。

4. 离散步长常量：`DT=0.025s`，输出降采样间隔=`1s`（40 个周期）。

3.6 适应性需求

调度器软件以源代码形式提供，通过 Makefile 支持本地编译和 ARM 交叉编译。用户可通过修改电流序列数组和定时周期参数适配不同的测试场景。

3.7 安全性需求

调度器软件的安全性需求如下：

1. 错误处理：clock_gettime 和 clock_nanosleep 调用失败时不应导致程序崩溃，应打印错误信息并尝试继续运行或安全退出。
2. 超时保护：当任务执行时间超过一个周期时，应检测并报告超时，防止调度器进入死循环或无限延迟。
3. 内存安全：不使用动态内存分配，所有变量使用栈或静态存储，避免内存泄漏。

3.8 保密性需求

调度器软件为嵌入式系统内部组件，不涉及用户权限管理、数据加密、审计日志或访问控制等保密性需求。软件在嵌入式平台内部运行，不与外部网络通信。

3.9 软件环境需求

调度器软件的运行环境需求如下：

1. 目标平台：ARM Linux（内核 4.1.15，单核处理器）
2. 编译器：GCC（支持 C99 标准）或 ARM 交叉编译工具链（arm-linux-gnueabi-hf-gcc）
3. 系统调用：需支持 clock_gettime(CLOCK_MONOTONIC) 和 clock_nanosleep 系统调用
4. 运行库：标准 C 库（libc）、数学库（libm）、POSIX 实时扩展库（librt）
5. 开发环境：支持在 x86 PC 上进行测试，目标平台为 ARM 嵌入式系统

3.10 计算机资源需求

3.10.1 计算机硬件需求

目标处理器：ARM 架构单核处理器，主频不低于 400MHz

内存：最低要求 32MB 系统内存

定时器精度：系统需支持 CLOCK_MONOTONIC 高精度定时器，精度不低于 1ms

3.10.2 计算机硬件资源使用需求

CPU 使用：以 25ms 周期运行时 CPU 占用率不超过 10%（含电池模型计算）

内存使用：运行时内存占用不超过 2KB（定时器状态+电流序列+统计变量）

栈使用：主循环调用深度不超过 5 层，栈使用量不超过 1KB

3.10.3 计算机软件需求

操作系统：嵌入式 Linux（内核版本 4.1.15 及以上）

运行库：libc（POSIX.1-2001 标准）、librt（POSIX 实时扩展）

构建工具：GNU Make、ARM 交叉编译工具链

3.10.4 计算机通信需求

调度器软件不需要网络通信功能，所有数据交互通过本地函数调用完成。输出通过标准输出打印，无计算机通信需求。

3.11 软件质量因素

调度器软件的质量因素要求如下：

1. 正确性：调度器应能正确驱动电池模型运行，输出结果与预期一致（放电电压下降、SOC 线性下降）。
2. 定时精度：连续运行 1000 个周期后总时间误差不超过 50ms，单个周期误差不超过 $\pm 2\text{ms}$ 。
3. 可靠性：软件应能长时间稳定运行（至少连续运行 2 分钟）无明显漂移或异常。
4. 性能：CPU 占用率不超过 10%（在 ARM 单核 400MHz 处理器上）。
5. 可维护性：代码结构清晰，模块化设计，定时器、电流序列和主循环分离。
6. 可移植性：仅使用 POSIX 标准 API，可在不同 Linux 平台间移植。

3.12 设计和实现约束

调度器软件的设计和实现约束如下：

1. 编程语言：仅使用标准 C99 和 POSIX API。
2. 内存管理：禁止使用动态内存分配（malloc/free），所有变量使用栈或静态存储。
3. 调度方式：使用 clock_nanosleep 绝对时间等待方式，不使用忙等待或 usleep。
4. 电池模型接口：不得修改电池模型的内部结构体，仅通过其提供的函数接口访问。
5. 非 RTOS：在标准 Linux 环境下运行，不依赖实时补丁或 RTOS。

3.13 人员需求

调度器软件开发人员需具备嵌入式 Linux 编程能力、POSIX 定时器使用经验和实时系统调试能力。

3.14 培训需求

本软件不需要特殊的培训需求，开发和使用人员可通过阅读本文档和源代码快速上手。

3.15 软件保障需求

软件以源代码形式交付，包含 scheduler.h/c、timer.h/c、main.c 和 Makefile。交付物支持本地编译和 ARM 交叉编译。

3.16 其他需求

无其他特殊需求。

3.17 验收、交付和包装需求（修改有关内容）

软件以源代码形式交付，包含 scheduler.h/c（定时器+主循环框架）、main.c（测试电流逻辑）和 Makefile（交叉编译用）。

3.18 需求的优先顺序和关键程度

高精度定时器功能和主循环调度功能为核心关键需求，时间漂移补偿功能为高优先级需求，电流序列管理和降采样输出为一般需求。

4 合格性规定

调度器软件的合格性方法如下表所示：

需求名称	需求的唯一标识	本文档中的章节号	合格性方法	合格性级别
高精度定时器	REQ-3.2.1	3.2.1	单元测试+精度测量	关键
时间漂移补偿	REQ-3.2.2	3.2.2	长时间运行测试	关键
电流序列管理	REQ-3.2.3	3.2.3	单元测试	一般
主循环调度	REQ-3.2.4	3.2.4	集成测试	关键

降采样输出	REQ-3.2.5	3.2.5	功能测试	一般
注：合格性方法： a) A-演示：对 CSCI（或部分 CSCI）进行直接可以观察的功能操作，不需要使用复杂或专用的测试设备； b) B-测试； c) C-分析； d) D-检查：对 CSCI 的编码、文档等进行直观检查等； e) E-特殊方法（如：专用工具、技术、过程、设施、验收限制）。 合格性级别： a) a-配置项：在 CSCI 中发现的质量问题； b) b-系统集成：在系统集成时发现的问题； c) c-系统：在系统中发现的质量问题； d) d-系统安装：在系统安装时发现的质量问题。				

5 需求可追踪性

本章建立调度器软件需求与系统需求之间的双向追踪关系。

表 5-1 需求的正向追踪性

《XXX 课题可行性研究报告》中的指标		软件需求	
指标内容	章节号	需求名称	章节号
25ms 周期驱动电池模型	总体目标	主循环调度功能	3.2.4
处理时间漂移（补偿执行开销）	总体目标	时间漂移补偿功能	3.2.2
周期性执行模型步进并记录输出	总体目标	降采样输出功能	3.2.5
若存在一对多则此形式表达			

表 5-2 需求的逆向追踪性

软件需求		《XXXXX 课题可行性研究报告》中的指标	
需求名称	指标内容	指标内容	章节号
主循环调度功能	25ms 周期驱动电池模型步进	25ms 周期驱动电池模型	总体目标
时间漂移补偿功能	绝对时间补偿消除漂移	处理时间漂移（补偿执行开销）	总体目标

软件需求		《XXXXX 课题可行性研究报告》中的指标	
需求名称	指标内容	指标内容	章节号
降采样输出功能	每秒输出电压、SOC 和功率	周期性执行模型步进并记录输出	总体目标

6 注释

术语定义：

clock_nanosleep：POSIX 高精度定时器系统调用，支持绝对时间和相对时间等待。

CLOCK_MONOTONIC：单调递增时钟，不受系统时间调整影响，适合用于定时测量。

时间漂移：由于任务执行开销和定时器精度限制导致的实际周期与目标周期的累积偏差。

降采样：将高频数据按固定间隔提取为低频数据的过程。

脉冲电流：在固定时间间隔内交替切换高低电流值的测试模式。

缩略语：

SRS - Software Requirements Specification（软件需求规格说明）

BMS - Battery Management System（电池管理系统）

DT - Discrete Time step（离散时间步长）

RTOS - Real-Time Operating System（实时操作系统）